Switchboard

A retrieval-gated agentic system for 500+ API tools

Repository: github.com/Kshitix03/switchboard
Domain: PayPal REST APIs · Build window: 2 days

1. Thesis and measured result

An LLM given 500 tool definitions performs worse than one given 6. Tool descriptions consume context, similar tools blur together, and the model picks wrong or hallucinates parameters. Prompt engineering does not fix this — it is a scaling wall. Tool selection at scale is a retrieval problem, not a prompting problem. If the model never sees more than 6–8 tools per turn, the prompt it receives stops growing with the registry, and accuracy stops being a direct function of registry size.

The registry scales from 108 real tools (107 PayPal operations across 11 official OpenAPI specs, plus `rag_search`) to 500, padded with real GitHub REST API operations tagged by source. Across that range:

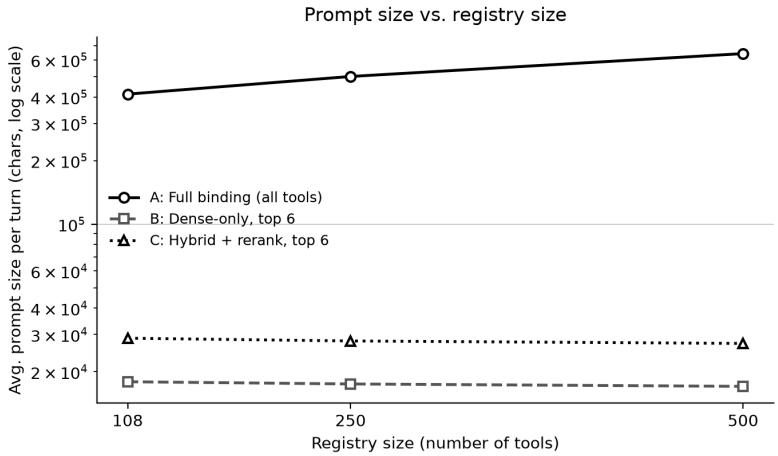

Figure 1. Prompt characters per turn as the registry scales 108→500. Full binding grows from ~413K to ~642K. Both retrieval-gated arms stay flat at ~17–29K regardless of registry size, because they only ever bind the top 6 retrieved candidates. Logarithmic y-axis.

Table 1. Retrieval metrics by registry size and arm (250-tool midpoint omitted for brevity; see `docs/RESULTS.md`).

Registry	Arm	Recall@6	Prompt chars	Latency p50 (ms)
108	A. Full binding	1.00 (trivial)	412,865	9.5
108	B. Dense-only, top 6	0.83	17,922	6.7
108	C. Hybrid + rerank, top 6	0.72	28,809	2465.8
500	A. Full binding	1.00 (trivial)	642,246	17.9
500	B. Dense-only, top 6	0.80	17,042	17.4
500	C. Hybrid + rerank, top 6	0.70	27,217	2429.7

What is measured at full scale is prompt size, not end-to-end accuracy. Section 5 explains that distinction and why it is load-bearing rather than a rounding error.

2. Agent structure

Switchboard is a LangGraph state machine, not a linear chain, because the real control flow has branches and loops a chain cannot express: a validation failure loops back to argument filling; a write-class tool must pause mid-turn for human approval and resume later; low-confidence retrieval must divert to a clarifying question instead of guessing.

LangGraph's `interrupt()` and SQLite checkpointer make pause-and-resume a first-class primitive rather than something bolted onto a pipeline.

```
plan ---> { chat | system_search | retrieve_tools }

retrieve_tools -> bind -> fill -> validate -> { fill (retry <= 2) | clarify | approve_gate }

approve_gate -> { dry_run | execute | rejected }

execute -> observe
```

`plan` classifies the turn (chat / retrieve-and-act / meta) in one structured-output call. `bind` loads full JSON Schemas only for retrieved candidates — schemas never enter context before this point. `validate` runs filled arguments through `jsonschema.validate` against the tool's real request schema; failures feed the validator's own error message back into a repair `fill` call, capped at 2 retries, costing zero additional user round-trips. A hard step cap (15) guards every conditional edge. Six distinct non-execution terminal states exist — `chat`, `system_search`, `clarify`, `dry_run`, `rejected`, `abort` — so a turn has many intentional ways to end besides success or crash.

3. Tool selection and routing

Cards vs. schemas — the mechanism behind the flat line

Every `ToolRecord` carries a short card (name, summary, description, keywords, utterances) that is embedded and BM25-indexed, plus a `schema_ref` pointing into a separate on-disk schema store. The full JSON Schema — often several KB with nested `$ref`/`allOf` chains in PayPal's real specs — never enters the vector index and never enters a prompt until `bind` loads it for the six or fewer survivors.

Hybrid retrieval, and an honest result

Dense (Qdrant, `gemini-embedding-001`, top 30) and sparse (BM25Okapi, top 30) run in parallel, fuse via reciprocal rank fusion (k=60, no weight tuning), then rerank with an off-the-shelf cross-encoder (`ms-marco-MiniLM-L-6-v2`) down to 6. A Phase 2 smoke test hit 10/10 on hand-picked near-miss pairs. The Phase 7 benchmark complicates that cleanly-positive story: hybrid+rerank recalls the correct tool *less* often than dense-only at every registry size (0.72 vs 0.83 at 108; 0.70 vs 0.80 at 500). Six spot-checked cases show the untuned cross-encoder — trained on general web query-passage relevance, not this domain — biased toward cards reading as direct action fulfillment over list/read/knowledge cards, in two cases dropping the correct tool (`plans.list`, `rag_search`) entirely. This is a real measured cost of "off the shelf, not tuned," not a hypothetical caveat.

Why utterances

Each record's 3–5 LLM-generated utterances are the single highest-leverage ingestion decision. Near-miss pairs (`invoices.create` vs `.send`; `subscriptions.suspend` vs `.cancel`) share almost no distinguishing path tokens but differ sharply in intent, which is exactly what utterances capture. The *C-minus* ablation (utterances removed) was cut under time pressure, so this claim is well-supported qualitatively but not benchmarked.

Filters and load-on-demand binding

`router.retrieve(query, filters)` accepts payload filters (`service`, `domain`, `operation`) applied to both legs before fusion. `plan` emits an `operation: read|write` filter when confident about mutation intent, halving the candidate pool before retrieval runs.

Two-step fill, not one

The first implementation asked one call to both pick a tool and fill its arguments, with a generic `{tool_id, args: object}` response schema. It reliably returned `args: {}` — structured decoding satisfies a loose `object` constraint with an empty object regardless of prose instructions. Splitting into (1) tool selection constrained to an enum of the 6

candidate ids, then (2) argument-filling using that tool's own flattened schema as the `response_schema`, fixed it immediately. Constrained decoding only works when the schema itself encodes the real structure.

One index, two renderers

`rag_search` is a normal registry entry (`domain=knowledge`) competing for retrieval slots like any PayPal tool; when selected it runs a real dense search over a docs corpus. `system_search` is not a competing entry — it is a second renderer over the identical `router.retrieve()` call, invoked when `plan` classifies a turn as "meta," rendering matching tools as prose instead of binding them as callables.

4. State management

`AgentState` is a `TypedDict` with three intended tiers, reported honestly, because only the first is fully built.

Table 2. State tiers, intent, and actual implementation status.

Tier	Intent	Status
Working state	Transient per turn: `plan`, `filters`, `candidate_tools`, `selected_tool`, `filled_args`, `validation_errors`, `retry_count`, `steps`	Fully implemented. Every node reads and writes it; the retry loop and step cap operate on it.
`entities`	Persist resolved references ("that customer") across turns without re-retrieval	Not implemented. Declared in the schema, but no resolution logic writes to it. The golden set's `RESOLVE:` markers are placeholders for exactly this.
`artifacts`	Large results return a handle plus summary, never raw JSON in context	Partial. Holds `bound_schemas` and `last_execution` (real, working), but the handle mechanism was never built — no mock response was large enough to force it.

Persistence is a `langgraph-checkpoint-sqlite SqliteSaver` keyed by `thread_id`. This is what makes `interrupt()` and `Command(resume=…)` work across a CLI prompt boundary, and what lets `system_search` answer "what is the status of my last request" from real state. A deliberately narrow JSONL trace log sits alongside the full OTel/Phoenix instrumentation to serve that one cheap lookup without querying Phoenix's storage — two trace stores for two different consumers, not redundancy.

5. Scalability

The measured headline is prompt size, not tool-selection accuracy, and that distinction is load-bearing. The first attempt at a full-binding accuracy metric ("is the expected tool the single nearest embedding neighbour across the whole registry?") turned out to be mathematically identical to dense-only's top-1 pick — nearest-neighbour-of-1 cannot change based on how many additional results are also retrieved. It was discarded once the mathematics was checked, rather than reported as a real divergence. Full binding's actual distinguishing failure mode — attention and context degradation when handed hundreds of schemas at once — is not a retrieval-ranking phenomenon and cannot be measured without large-context generation calls at every registry size, which this build's free-tier quota did not support.

A small end-to-end validation sample (n=15, 108-tool tier, real LLM calls through the production `_select_tool` path) shows the zero-cost retrieval proxy *under*estimating true accuracy by 26 points: proxy 0.47 versus real 0.73. Given the same 6 candidates, the model's own reasoning recovers from several retrieval ranking mistakes. The proxy is therefore a conservative lower bound, not a faithful accuracy estimate.

So the honest scaling story is three separate facts: prompt size is flat and measured at full scale; tool-selection accuracy is real but sampled only at the smallest tier; and the gap between them is disclosed rather than papered over with an extrapolated curve.

What breaks next: registry ingestion cost is linear in registry size (each domain needs an enrichment pass — this build hit daily quota limits repeatedly enriching ~110 tools). Rerank cost is bounded by fusion width (30+30 minus overlap), not registry size, but a naive implementation reranking the whole registry would not scale. And cross-service planning is absent: `fill` picks exactly one tool per turn, so a request spanning two providers would need planning this design does not yet do.

6. Error handling

Handled by failure class, not one generic retry.

Table 3. Failure classes, designed response, and implementation status.

Failure	Designed response	Status in this build
Retrieval miss, low rerank score	Widen once, then `clarify` — never guess	Implemented; score threshold routes straight to clarify. The widen-once retry was not built.
Schema validation fails	Repair loop with validator errors fed back, cap 2	Implemented and tested; zero extra user round-trips.
400 / 422 from API	Feed error body back once, then escalate	Not applicable — execution is fully mocked.
401 / 403	Never retry, escalate immediately	Not applicable — mocked.
429 / 5xx	Backoff with jitter, circuit breaker per service	Not applicable to the mocked agent, but this build's own tooling hit exactly this class against the real Gemini API and needed real backoff and a mid-build model swap.
Ambiguous entity	Resolve via a read tool first, or ask	Not implemented — see section 4.
Loop without progress	Hard step cap, abort with explanation	Implemented and tested (cap = 15).

Idempotency is the payments-specific point. Every write derives a deterministic key — `sha256(trace_id : tool_id : sorted(args))` — so a retried "send $50" carries the same key across a graph resume rather than a fresh one. A duplicate write, not a duplicate read, is the failure that actually matters in this domain.

The approval gate calls `interrupt()` before any `operation=="write"` or `risk=="high"` tool. The `--dry-run` flag short-circuits before the interrupt is ever raised, so a dry run can never pause waiting on a human, and no write reaches `execute` unapproved.

7. Framework choices

- **LangGraph over CrewAI.** Explicit cyclic control flow, `interrupt()`-based approval, and durable checkpointing are load-bearing. CrewAI's role abstraction hides control flow — the wrong trade when a wrong turn moves money.
- **LangGraph over plain LangChain.** Retry, clarify, and approval are branches and loops; a chain is linear.
- **Qdrant embedded, not Docker.** No Docker dependency for a 2-day build. Trade-off: the index rebuilds each process start, mitigated by caching embeddings to disk by content hash separately from the ephemeral collection.
- **Gemini over OpenAI or Anthropic.** A quota-driven choice that cost real time: `gemini-2.5-flash` returned 404 as "no longer available to new users" despite showing console quota; the working alias hit its 20 requests/day cap mid-build and had to be swapped. A free-tier key shaped several architecture decisions more than anticipated.
- **Native structured output over ReAct text parsing.** Fewer parse failures and provider-enforced schemas — but only once the schema encodes real structure (section 3).
- **Phoenix over LangSmith.** Open source, OTel-native, runs locally; `cli.py` launches it automatically. Every node is span-wrapped with funnel candidate ids, rerank scores, token counts, and retries.

- **DSPy.** Named as future work for optimising planner and fill prompts against the golden set — not used here.

8. Limitations and what I would build next

Named plainly: a disclosed weakness with a measurement attached is worth more than a claim of none.

1. **The golden set is entirely AI-generated, not hand-written.** A direct, disclosed deviation from the specification's explicit instruction. The real risk — a model generating both the routing system and its own eval queries can correlate blind spots — is named rather than hidden. Next: a genuinely hand-written 50–60 query set, authored blind to this system's behaviour.
2. **Retrieval gating's inherent hard failure mode.** If the correct tool misses the top 6, this agent cannot recover the way a full-binding agent theoretically might. Mitigated by measuring recall@6 directly and routing low-confidence turns to `clarify`, but not eliminated.
3. **The untuned reranker measurably hurts recall versus dense-only** at every registry size. Next: fine-tune on this domain's query/tool pairs (the generated utterances are a ready-made training signal), or make reranking conditional on whether it improves the candidate set.
4. **Entity resolution and artifact handles are unimplemented** despite being in the state schema — the single largest gap between the design as specified and the system as built.
5. **Accuracy versus scale was never measured with real LLM calls at every tier** — only a 15-query sample at the smallest tier, due to free-tier quota. Prompt size is measured at full scale.
6. **Padding is cross-domain** (GitHub, not another payments API), which likely understates real degradation, since unrelated tools rarely become plausible distractors. Next: pad with same-domain APIs for a harder test.
7. **A live routing bug, disclosed rather than silently fixed:** "how long do buyers have to open a dispute" should route to `rag_search` but picks `disputes.list` — exactly the knowledge-versus-action confusion the golden set was designed to probe.
8. **Accepted cuts.** Execution is fully mocked (`execute_live()` raises rather than silently no-op'ing); multi-step tool sequencing is out of scope; and the benchmark ran once per configuration, not averaged over seeds.

Full report, decision log, and benchmark methodology at github.com/Kshitix03/switchboard — see `REPORT.md`, `DECISIONS.md`, and `docs/RESULTS.md`.